



SIH 26171

On-device Visual Perception for Lightweight Browser Agents

Research, Architecture & 30-Day Engineering Blueprint

DECISION IN ONE LINE Build a Chrome MV3 edge-cloud browser agent that uses structure first, performs real local visual/OCR perception only where needed, detects and tokenizes sensitive data before networking, sends a minimized semantic UI representation plus privacy-safe crops only on escalation, and treats the cloud planner as untrusted: it proposes constrained actions; the browser validates, re-grounds, executes, and verifies them locally.

Constraint	Design consequence
Six first-year CSE students / ~30 days	Optimize for a working vertical slice, measurable scoring, and integration risk - not model novelty.
Privacy before network	No raw screenshot, raw DOM dump, raw form values, or secret-bearing telemetry may leave by default.
Real local visual processing required	Include a genuine browser-side CV path (OCR + lightweight detector/grounder), not DOM-only automation.
Scored on accuracy, privacy, resources, latency	Every subsystem needs benchmark hooks from Day 1.
Unseen finale use cases	Avoid site-specific selectors; fuse structure, pixels and re-grounding.

Prepared from the team research specification and current public technical/research sources available through 24 Aug 2026.

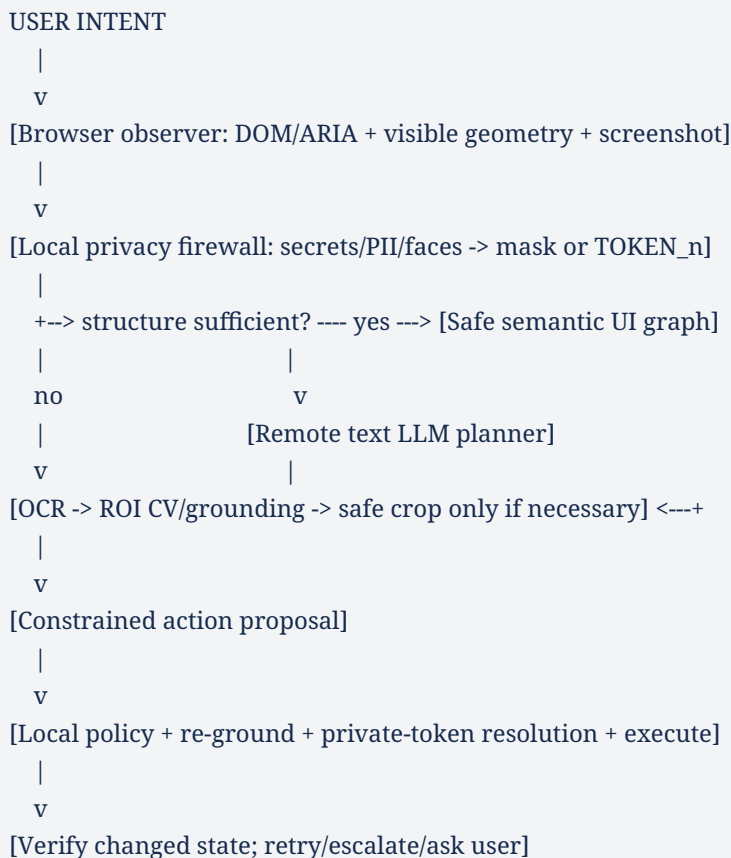
Evidence labels used: [REQ] official/problem boundary supplied by team; [DOC] official technology documentation; [PAPER] published/preprint research; [IMPL] existing implementation; [INF] engineering inference; [DESIGN] proposed design; [EXP] experiment required.

1. One-Page Executive Understanding

What ISRO is asking for is not merely a browser extension and not merely an AI model. It is a privacy-preserving edge-cloud browser agent: the browser must perceive enough of the current UI locally, identify sensitive information locally, sanitize/minimize that information before any network request, ask remote intelligence for high-level reasoning when useful, and execute the returned action locally. The supplied problem boundary explicitly requires browser-side client code, real local visual processing, local PII detection, pre-network sanitization, centralized LLM/VLM reasoning, local action execution, and a complete end-to-end task. [REQ]

The core engineering tension is that pixels contain information structure misses, but pixels are expensive and privacy-heavy; DOM/accessibility structure is cheap and precise, but can omit canvas/PDF/image/icon semantics and can itself leak hidden/private data. The winning architecture therefore should not choose “DOM or vision.” It should use the cheapest trustworthy signal that can answer the current question, then escalate perception only when uncertainty or task requirements justify it. [INF][DESIGN]

WINNER Adaptive Structure + Local Vision + Privacy Vault + Remote Planner + Local Authority



Rank	Architecture	Why
------	--------------	-----

#1	Adaptive hybrid structure + vision edge-cloud	Best balance of compliance, privacy, latency, generalization, resource use and buildability.
#2	Vision-first protected semantic agent	Stronger on canvas/image-heavy pages; higher local compute and integration risk.
#3	Screenshot-first local redaction + cloud VLM	Fastest compliant fallback; simplest demo, but larger privacy surface/bandwidth and weaker resource score.

FIRST 48 HOURS Do not begin by integrating a large GUI model. Prove one vertical slice: capture -> local OCR/PII -> irreversible sanitization -> inspect outbound payload -> remote planner returns JSON action -> local allowlist/re-ground -> execute -> verify. Then benchmark and replace weak pieces one at a time.

2. Official Problem Deconstruction

The team specification fixes five scored dimensions: visual-context accuracy (25%), sensitive/PII precision-recall (20%), redaction precision (20%), client resource utilization (20%), and end-to-end latency (15%). It also states that finale use cases may be unseen. [REQ]

Requirement	Engineering interpretation	Proof in demo
Browser-side extension/client	MV3 extension owns observation, privacy gate and authority.	Show extension contexts and local logs.
Real local visual processing	At least OCR and/or lightweight visual detector/grounder executes in browser.	Disconnect server or inspect network while local model still returns boxes/text.
Local sensitive detection	PII/secrets detected before serialization.	PII overlay and detector trace.
Sanitize before network	Network layer only accepts SafeContext schema.	DevTools Network payload contains placeholders, not originals.
Central LLM/VLM	Server plans over protected context.	Server request/response visible.
Local action execution	Remote side cannot directly control browser.	Action passes local validator and re-ground step.
End-to-end task	Observe -> protect -> plan -> act -> verify.	60-90 second live workflow.

Product answer: you are building a combination - a browser extension, local perception stack, privacy firewall/vault, remote planning service, constrained browser-action engine, and benchmark

harness. Training a new foundation model is optional and should not be on the critical path.

[DESIGN]

3. User/Product Story

Example: a user opens a government scholarship portal containing their name, email, Aadhaar-like identifier, uploaded document preview and several unlabeled icon buttons. They ask: “Find the application status and open the document upload step.” The agent needs visible labels, layout, current step and actionable controls; it does not need the exact identifier or email. Locally, the extension extracts visible DOM/ARIA, captures the viewport, OCRs visual-only text if needed, detects PII and replaces values with typed placeholders. It sends a safe UI graph such as {role:"button", name:"Upload document", id:"e17", box:[...]}, plus task text. The remote planner returns a constrained proposal such as CLICK(e17). The browser re-checks that e17 still exists, is visible, matches the expected semantics, and is allowed; then it clicks and verifies the step changed. If the button is canvas-only, local visual grounding is escalated for that region. [DESIGN]

4. The Real Engineering Problem

Naive approach	Why it fails
Send raw screenshots to cloud	Directly violates the privacy objective; screenshots can contain secrets, messages, faces and documents.
Redact screenshot only	Missed regions still leak; blur can preserve recoverable structure; DOM/telemetry may leak separately; full images waste bandwidth.
DOM only	Fails on canvas, rendered PDFs, image text, icon-only controls, virtualized/visual semantics; also risks hidden DOM leakage.
Accessibility only	Useful semantics, but not a guaranteed pixel-equivalent view and not a complete solution for visual content.
OCR only	Text is not enough for icons, visual grouping, charts, non-text controls and robust grounding.
Large local VLM	Model download, memory, latency and browser operator support are risky under a 30-day student build. Ferret-UI Lite is 3B parameters even though it is “small” by GUI-agent standards. [PAPER R6]
Everything local	Can improve privacy but increases compute/model complexity and is not necessary to satisfy the stated centralized reasoning path.
Remote LLM only	Cannot safely or reliably ground actions without local observation and privacy filtering.
Selenium/Playwright as product	Great for testing, but the challenge is an in-browser privacy-preserving perception/agent system, not external automation.

Conceptually, architecture selection is a constrained multi-objective optimization: maximize task success x visual accuracy x privacy x reliability x generalization while minimizing client compute x latency x bandwidth x cost x integration risk. There is no single scalar optimum until the competition weights and feasibility constraints are applied. [INF]

5. Complete Solution-Space Map

Family	Flow	Upside	Primary risk	Verdict
A Screenshot-first privacy agent	Local screenshot CV/PII -> sanitized screenshot -> cloud VLM	Strong visual completeness; simple	High privacy surface, bandwidth, redaction FN risk	Fallback
B Structure-first agent	DOM/ARIA -> filter -> text LLM; pixels only exceptional	Fast, cheap, precise controls	Not sufficient alone for explicit local vision requirement / visual-only UI	Component, not final
C Local vision-first semantic agent	Screenshot -> local GUI model -> safe semantic representation	Strong pixel independence	Heavier models and browser integration	Alternative
D OCR-centric	Screenshot -> OCR boxes -> PII -> planner	Browser-feasible, measurable	Misses non-text semantics	Specialist path
E Specialist multi-model	OCR + face + PII + control detectors	Modular, replaceable	Pipeline complexity and duplicate work	Useful selectively
F Hybrid structure + vision	DOM/ARIA + OCR/CV fusion -> privacy -> planner	Balanced coverage	Fusion engineering	Strong
G Adaptive/cascaded	Structure -> OCR -> ROI vision -> safe crop	Compute only when needed	Needs uncertainty policy	Best
H Edge-cloud collaboration	Local perception/privacy; cloud planning	Matches challenge shape	Trust boundary must be strict	Best
I Mostly on-device	Local perception + local planner	Privacy/offline	Compute/model burden	Not critical path
J Semantic screenshot/SoM	Overlay stable IDs/boxes then send sanitized visual	Planner grounding aid	Still image-heavy; overlays can obscure	Optional escalation

6. Architecture-by-Architecture Decision Matrix

Architecture	Compliance	Privacy	Client cost	Latency	Generalization	30-day feasibility	Score potential
Screenshot-first	High	Medium	Medium	Medium	High	High	Medium-High

Structure-first only	Medium	High	Low	Low	Medium	High	Medium
Vision-first semantic	High	High	High	Medium-High	High	Medium-Low	High if it works
OCR-centric	High-ish	High	Medium	Medium	Medium	High	Medium
Specialist multi-model	High	High	Medium-High	Medium	High	Medium	High
Hybrid structure+vision	High	High	Medium	Low-Medium	High	Medium-High	High
Adaptive cascade	High	Very high	Low-Medium	Low-Medium	High	Medium	Very high
Mostly on-device	Medium-High	Very high	Very high	Variable	High	Low	Risky

The ranking is an engineering judgment, not a published benchmark. It must be validated on the team hardware and unseen-site test set. [INF][EXP]

7. Perception Design Space

The browser should separate “what structure already knows” from “what only pixels can reveal.” DOM/ARIA is best for visible text, roles, states, input types, labels, element identity and exact executable targets. Pixels are necessary for canvas content, image text, visual-only icons, charts, rendered documents, appearance/layout relationships and cases where DOM semantics are absent or adversarial. [INF]

Signal	Use it for	Do not trust it for
DOM	Visible text, roles, values after privacy filtering, element identity, selectors	Hidden/off-screen data, visual truth, canvas
ARIA/accessibility semantics	Role/name/state and semantic control discovery	Complete visual coverage
Screenshot	Ground truth of what is rendered	Privacy-safe transport by default
OCR	Text in pixels/PDF/canvas/images; coordinates	Icon meaning and full UI semantics
Light CV detector	Faces, documents, QR/barcodes, UI regions	High-level task planning
GUI grounder	Text/icon-to-point/box when structure fails	Always-on perception before it is benchmarked
Visual differencing	Verification and changed-region targeting	Semantic success by itself

CASCADE POLICY 1) Structure if sufficient. 2) OCR visible ROI if missing text. 3) Lightweight local vision/grounding if target is still unresolved. 4) High-resolution ROI only for ambiguous regions. 5) Send a privacy-safe crop to remote VLM only when the planner truly needs pixels. ScreenSpot-Pro reports that reducing the search area materially improves GUI grounding, supporting ROI/cascaded search as a serious design direction. [PAPER R5]

8. Privacy Detection Architecture

Privacy is not a single NER model. The strongest practical system is layered because secrets and PII have different signatures and locations. [DESIGN]

Layer	Mechanism	Examples	Priority
Deterministic	Regex + checksum/format validation	email, phone, card-like numbers, IDs, API keys, JWT-like tokens	USE
DOM semantics	input type/name/autocomplete/labels; password fields	password, OTP, account fields	USE
Local NER	Small token-classification model over visible text	person, organization, address-like entities	TEST
OCR + same detectors	Run privacy stack over pixel-derived text	image/PDF/canvas PII	USE
Visual detectors	face / document / QR regions	faces, ID cards, signatures, QR	TEST
Policy	site/user/task sensitivity and deny rules	banking, health, government portals	USE
Fusion	union for high-risk classes; confidence-aware review	reduce false negatives	USE

GUIGuard (2026) frames privacy recognition, protection and task execution as separate stages and reports privacy recognition as a major bottleneck; this is strong evidence against treating privacy as a cosmetic blur step. [PAPER R7]

9. Sanitization Mechanisms

Mechanism	Privacy	Utility	Recommendation
Blur	Weak-Medium	High	Avoid for secrets; visual demo only
Pixelation	Medium	Medium-High	Avoid as sole irreversible protection

Black box	High if box is correct	Medium	Use for images/secrets when semantics unnecessary
Irreversible replacement	High	High	Use for textual PII: [EMAIL_1], [PHONE_1]
Deletion	Very high	Low-Medium	Use for irrelevant private content
Cropping	High	Variable	Use for safe ROI minimization
Structural filtering	Very high	High	Default for DOM/ARIA payload
Generalization	High	High	e.g. exact age/date -> category when task permits
Local private-value vault	High	Very high	Use for values needed later but never remotely disclosed

The private-value vault maps placeholders such as [EMAIL_1] to actual values only on-device. The cloud can request TYPE(target, "[EMAIL_1]"); the browser validates target type/domain/user intent, resolves the token locally, types the real value, and never returns it to the planner. This preserves task utility while minimizing disclosure. [DESIGN]

10. What Should Cross the Network?

Representation	Privacy surface	Bandwidth	Planner utility	Default?
Full sanitized screenshot	Medium	High	Very high for VLM	No
Sanitized crop	Low-Medium	Low-Medium	High for local ambiguity	Escalation
Filtered DOM	Medium if over-collected	Medium	High	No raw dump
Filtered ARIA	Low-Medium	Low	High	Useful input
OCR text + boxes	Medium	Low-Medium	High	Only sanitized
Structured element list	Low	Low	High for text LLM	Yes
Semantic UI graph	Low	Low	Very high	Yes
Combination: graph + safe crop	Low-Medium	Low-Medium	Very high	Best escalation

MINIMUM REMOTE CONTEXT User goal + page identity (sanitized) + visible actionable elements with stable local IDs, roles, safe names, states and coarse geometry + relevant non-action text + prior action outcome. Add a safe visual crop only when semantic context cannot resolve the task.

11. Current Browser/ML Technology Reality

Chrome MV3 uses service workers for the background context; service workers lack DOM access, while the Offscreen API can create hidden extension documents for DOM/window-dependent work. Chrome documents Offscreen as available from Chrome 109+ in MV3. [DOC R1] Content scripts normally run in an isolated world; Chrome warns that hostile pages can still manipulate DOM assumptions, so local validation cannot blindly trust page-provided structure. [DOC R2][DOC R4]

`chrome.tabs.captureVisibleTab()` captures the visible area of the active tab and Chrome caps it at two calls per second, explicitly noting capture is expensive. That makes continuous full-screen screenshot loops a poor default and supports event-driven/ROI capture. [DOC R3]

ONNX Runtime Web supports WASM and GPU paths; its current docs recommend WASM for very lightweight models and WebGPU for more compute-intensive models. Its support table shows WASM broadly available, while WebGPU support is strongest on Chromium and operator coverage can differ by execution provider. [DOC R8][R9] Transformers.js runs models in-browser, defaults to WASM CPU, supports WebGPU and quantized dtypes, and exposes NLP and CV pipelines. [DOC R10]

Tesseract.js v7 is an Apache-2.0 browser/Node wrapper around a WebAssembly port of Tesseract, making it a low-friction OCR baseline. Its own docs recommend reusing a worker and document performance tradeoffs. [IMPL R11]

Technology	Decision	Why
Chrome MV3 + TypeScript	USE	Competition focus; modern extension architecture; local authority.
WXT/Plasmo-like framework	TEST	Can accelerate scaffolding, but raw MV3 is acceptable if framework friction appears.
ONNX Runtime Web	USE	General ONNX runtime with WASM/WebGPU fallback.
Transformers.js	TEST	Fast path for compatible NER/CV models; built on browser-friendly runtime patterns.
Tesseract.js	USE baseline	Simple OCR baseline; replace only if benchmarked weakness matters.
WebGPU	TEST + fallback	Performance upside, but do not make demo correctness depend on it.
WASM	USE fallback	Broad compatibility and predictable baseline.
OpenCV.js	OPTIONAL	Preprocessing, crops, thresholding, differencing; avoid overuse.

12. Model Candidates: What to Test, Not Assume

Task	Candidate class	Why	Gate
OCR	Tesseract.js v7 baseline	Runs in browser via WASM; simple integration.	CER/WER + latency on UI text
Text PII/NER	small ONNX token classifier + deterministic rules	Rules catch high-confidence secrets; NER expands semantic classes.	class recall, bundle size, p95
Face detection	small browser-compatible detector (MediaPipe/ONNX class)	Visual PII needs region boxes.	recall on small/partial faces
GUI grounding	GoClick 230M candidate if weights/runtime/license permit	2026 paper targets lightweight GUI grounding and edge-cloud collaboration. [PAPER R12]	actual ONNX/browser conversion + latency
GUI grounding fallback	DOM/ARIA IDs + OCR box matching	Much lower integration risk.	unseen-site grounding accuracy
Large on-device GUI agent	Ferret-UI Lite 3B	Research reference, not critical path. [PAPER R6]	likely reject on resource/integration cost

IMPORTANT Published model benchmark numbers are not your expected project performance. Any candidate affecting the final design must be tested on your browser build, target laptop(s), viewport and adversarial pages.

13. Remote Planner

Default to a text LLM over the safe semantic graph. Use a VLM only on escalated safe crops. This reduces image tokens, bandwidth and privacy surface while retaining visual reasoning where pixels genuinely matter. [DESIGN]

Planner output	Schema rule
CLICK	{type:"CLICK", target_id:"e17", expected:{role:"button", name_hash:" ..."}}}
TYPE	{type:"TYPE", target_id:"e8", value:"[EMAIL_1]"} - never raw secret
SCROLL	bounded direction/amount or container ID
SELECT	target ID + safe option label/token
WAIT	bounded milliseconds / condition
NAVIGATE	URL must pass origin/policy checks

ASK_USER	required for ambiguous or high-impact actions
COMPLETE	summary only; no hidden side effects

Reject free-form JavaScript, arbitrary selectors from the cloud, shell commands, or “click coordinates” without local re-grounding. The remote model proposes intent; the local browser owns authority. [DESIGN]

14. Action Execution & Re-grounding

Targeting method	Strength	Weakness	Use
Local element ID -> live DOM node	Precise, cheap	Stale after rerender	Primary
ARIA role/name	Semantic	Duplicates / dynamic labels	Re-ground aid
CSS/XPath selector	Deterministic	Brittle; planner should not invent	Internal fallback
Coordinates	Works on visual-only UI	Stale with scroll/layout	Only after local visual re-ground
Visual grounding	Handles canvas/icon UI	Compute + uncertainty	Escalation

Immediately before execution: re-read page epoch -> find target -> confirm visible/enabled -> compare role/name/geometry fingerprint -> check action policy -> resolve local private token if any -> execute -> verify DOM/URL/visual delta. If the page changed, discard the proposal and re-plan rather than clicking stale coordinates. [DESIGN]

15. Security Architecture

AgentDojo demonstrates that LLM agents processing untrusted data are vulnerable to prompt injection, and later web-agent work shows indirect prompt injection can be embedded in webpage content. Therefore webpage text must be treated as untrusted data, not instructions, and remote planner output must be treated as untrusted proposals. [PAPER R13][R14]

Threat	Control
Malicious webpage / prompt injection	Separate user intent from page data; tag provenance; no page text may alter system policy; action allowlist.
Malicious/incorrect planner	Constrained schema; local policy engine; confirmation for sensitive actions; no direct secret access.
Private token exfiltration	Tokens are opaque; resolver never returns underlying value; deny typing token into unexpected origin/field.

Stale action	Page epoch + target fingerprint + re-ground immediately before action.
Hidden DOM leakage	Visible-only extraction by default; strip hidden/off-screen/value/data-* unless explicitly required and safe.
Network logs/telemetry	Log SafeContext only; redact server errors; disable request-body analytics.
Cross-origin frames	Inject only with explicit permissions; frame provenance; if inaccessible, rely on rendered pixels and user confirmation.
Renderer compromise	Cannot fully solve in app layer; minimize privileges and secret lifetime.

SECURITY PRINCIPLE Remote intelligence, local authority. The cloud may suggest; only the local policy engine may authorize and execute.

16. Top Three Architectures

#1 - Adaptive Hybrid Edge-Cloud Agent

Structure-first observation + local privacy firewall + OCR/CV escalation + minimized semantic graph + remote text planner + safe-crop VLM fallback + local policy/re-ground/execution/verification. It wins because it aligns every expensive/privacy-heavy operation with demonstrated need. [DESIGN]

#2 - Vision-First Protected Semantic Agent

Local screenshot understanding produces a protected element graph before cloud planning. It can outperform #1 on canvas-heavy or visually rich interfaces where DOM/ARIA is poor, but only if a lightweight GUI model actually meets browser latency/memory constraints. GoClick's 230M-parameter direction is promising research evidence, not yet proof of browser deployment feasibility for this team. [PAPER R12][EXP]

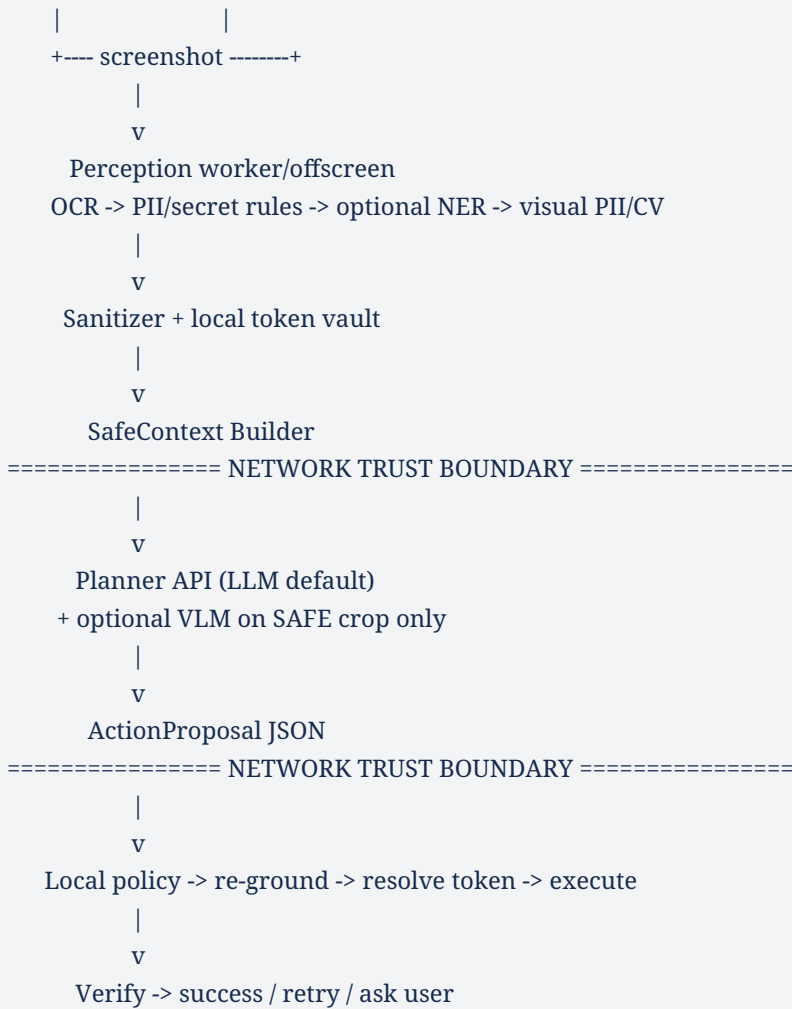
#3 - Screenshot-First Redaction + Cloud VLM

Capture viewport -> local OCR/PII/face detection -> irreversible mask -> cloud VLM -> local action validator. It is the safest schedule fallback because the architecture is easy to explain and demo. It loses on privacy surface, bandwidth and possibly resource/latency metrics compared with #1. [DESIGN]

17. Winning Architecture - Implementation Blueprint

EXTENSION TRUST ZONE

Content script -> visible DOM/ARIA collector -> element registry



Component	Input	Mechanism	Output	Runs where	Fallback	Metric
Observer	page	DOM/ARIA + viewport capture	raw local scene	browser	screenshot only	coverage/ms
Element registry	visible nodes	stable local IDs + geometry	element graph	browser	role/name match	grounding acc
OCR	ROI image	Tesseract baseline	text boxes	browser worker	DOM text	CER,p95
Privacy detector	text/boxes/ metadata	rules + optional NER + visual detectors	sensitive spans/regions	browser	fail-closed policy	P/R by class
Sanitizer/vault	scene + detections	delete/mask/ tokenize/minimize	SafeContext	browser	black-box crop	leakage rate
Planner	goal + SafeContext	LLM; VLM only safe crop	ActionProposal	server	rule planner demo	task success
Policy/re-ground	proposal + live page	schema + origin + fingerprint	approved action	browser	ask user	unsafe-action rate

Executor	approved action	DOM action / local point	state change	browser	re-plan	execution success
Verifier	before/after state	URL/DOM/visual delta	success signal	browser	user confirmation	false success rate

18. Exact Product Definition

- Installed on the PC: one Chrome MV3 extension. No native daemon in the MVP.
- Inside the browser: observer, screenshot capture, OCR/CV worker, privacy detector, sanitizer, token vault, action policy, executor, verifier and debug/benchmark panel.
- Local models: OCR baseline; optional small NER; optional visual PII detector; optional lightweight GUI grounder only after benchmark gate.
- Server: thin authenticated planner API with strict request/response schemas, no persistent raw context logging, and provider adapter for text LLM / optional VLM.
- Server receives: sanitized user goal, safe element graph, relevant safe text, action history, and only privacy-safe crops when escalated.
- Server never receives by default: raw screenshot, raw DOM dump, passwords, OTPs, API/session tokens, private-vault values, hidden form values or unfiltered telemetry.
- Remote AI: chooses next high-level action and explains uncertainty; it does not execute browser code.
- Browser: validates, re-grounds, resolves local tokens, executes and verifies.
- User sees: side panel with goal/status, local privacy detections, “cloud receives” inspector, and confirmations for high-risk actions.

Repository/module	Contents
extension/	MV3 manifest, content script, service worker, side panel, messaging
perception/	OCR adapter, CV adapters, ROI manager, model loader
privacy/	rules, NER adapter, visual privacy, sanitizer, vault, policy
protocol/	SafeContext and ActionProposal schemas + validators
server/	planner API, model-provider adapters, zero-sensitive-log middleware
execution/	element registry, re-grounding, action executor, verifier
bench/	datasets, scenarios, metrics, profiler, adversarial cases
demo/	controlled pages + scripts + judge walkthrough
docs/	architecture, threat model, experiment log, claims/evidence table

19. Tool / Technology Matrix

Area	Choice	Status	Reason
Language	TypeScript	USE	One language across extension/server adapters where possible.
Extension	Chrome MV3	USE	Primary target; minimize multi-browser risk.
UI	Vanilla/React side panel	TEST	Use React only if team is already fast with it.
Local ML	ONNX Runtime Web	USE	WASM baseline + WebGPU acceleration.
HF models	Transformers.js	TEST	Convenient compatible pipelines and quantization.
OCR	Tesseract.js v7	USE baseline	Browser-native WASM baseline.
CV preprocessing	Canvas/OffscreenCanvas; OpenCV.js optional	USE/OPTIONAL	Avoid heavy dependency unless it improves metrics.
Backend	FastAPI or lightweight Node API	USE	Pick team-familiar option; schemas first.
Planner	Hosted strong text LLM	USE	Fastest path to robust planning; VLM only escalation.
Schema	JSON Schema/Zod/Pydantic	USE	Hard boundary between planner and executor.
Testing	Vitest/Jest + Playwright	USE	Playwright for repeatable test pages, not as product executor.
Profiling	Performance API + Chrome DevTools	USE	p50/p95, memory, network bytes.
Git	GitHub PRs + integration branch	USE	Continuous vertical-slice integration.
Kubernetes/RAG/multi-agent swarm	None	AVOID	No scored value for 30-day MVP.

20. Human Work vs AI Coding Tools

AI coding/research tools should accelerate	Humans must own
Extension scaffolding, repetitive adapters, type definitions, tests, mock pages, benchmark scripts, documentation drafts, refactors, debugging hypotheses	Architecture, privacy boundaries, threat model, experiment design, model selection, metric interpretation, integration decisions, security review, final claims

Generate code only behind explicit interfaces and tests	Never accept “it runs” as evidence; inspect network, metrics and failure behavior
Use separate prompts per bounded module	One human integration owner controls protocol changes

Recommended workflow: define interface -> ask AI for implementation + tests -> human reviews threat/privacy assumptions -> run tests/benchmarks -> log result -> merge only if the vertical slice still works. Do not ask an agent to “build the whole project.” [DESIGN]

21. Minimum Real Prototype & 60-90 Second Demo

Time	Judge sees	System proves
0-10s	Open test portal with visible email/phone/ID and one visual-only control	Realistic sensitive screen
10-25s	User asks task; local overlay highlights PII and OCR/vision box	Local visual processing + local PII
25-40s	Open DevTools Network; outbound JSON shows [EMAIL_1], safe elements, no raw screenshot	Sanitization before network + minimization
40-55s	Server returns CLICK/TYPE proposal	Central reasoning
55-70s	Local validator shows target fingerprint and resolves token locally	Remote intelligence, local authority
70-90s	Browser executes; verifier confirms state change; benchmark panel shows latency/bytes	End-to-end success + measurement

DEMO RULE Do not rely on a fake “cloud received” visualization. Show the actual request payload in DevTools or a server-side request inspector that logs only sanitized fields.

22. Execution Order - First 48 Hours to Day 30

Phase	Primary tasks	Exit criteria	Fallback
First 24h	Repo + MV3 shell; capture visible DOM/ARIA + screenshot; define SafeContext/ActionProposal; local regex PII; one mock page; network inspector	One click task can observe -> sanitize -> send -> receive mock action	No ML yet; deterministic vertical slice
24-48h	Tesseract local OCR; map OCR boxes to PII; real planner API;	Real end-to-end task with no raw	Use DOM text if OCR slow; keep

	local CLICK/TYPE validator; verification; baseline profiler	PII in network	OCR visible as compliance proof
Days 3-5	Element registry; token vault; hidden-DOM filtering; page epoch/re-ground; 5 test pages; metrics harness	Repeatable benchmark suite and safe token typing	Ask-user on ambiguous target
Week 2	Optional small NER; face/QR/document detector experiments; ROI manager; canvas/PDF cases; security tests	Each added model earns its place on measured metric	Drop model if p95/memory/accuracy tradeoff is poor
Week 3	Adaptive cascade; optional GUI grounder experiment; safe-crop VLM escalation; unseen sites; failure recovery	Generalization and recovery demonstrated	Stay DOM+OCR+safe crop
Week 4	Freeze architecture; optimize bundle/model cache; adversarial privacy; judge demo; evidence/claims; backup build/video	Stable 60-90s demo + scorecard + fallback build	Switch to screenshot-first redaction fallback if integration regresses

23. Six-Person Team Allocation

Owner	Primary responsibility	Must integrate daily with
1 - Architecture/Integration	schemas, branch gate, end-to-end vertical slice, performance budget	everyone
2 - Browser Systems	MV3, capture, DOM/ARIA, element registry, side panel	1,5
3 - Local Perception	OCR, ROI, CV/grounder experiments, model runtime	2,4,6
4 - Privacy/Security	PII rules/NER, sanitizer, vault, threat tests	2,3,5
5 - Backend/Agent	planner API, structured actions, provider adapters, zero-sensitive logging	1,4,2
6 - Evaluation/Demo	test pages, datasets, metrics, profiler, unseen/adversarial suite, demo script	all

Anti-silo rule: every teammate must merge into the working vertical slice at least every 24-48 hours. Interfaces are owned centrally; nobody may change SafeContext or ActionProposal without integration review. [DESIGN]

24. Benchmark Plan Mapped to SIH Weights

Official dimension	Measure	Method
Visual accuracy - 25%	element grounding accuracy, text/box accuracy, visual-only control success	annotated screenshots + unseen UI set; stratify DOM-rich vs canvas/PDF/image
PII P/R - 20%	precision, recall, F1 per class; especially secret-class recall	annotated spans/boxes across DOM + OCR + images; report macro and per-class
Redaction precision - 20%	sensitive pixels/spans covered; utility pixels/text preserved; leakage count	compare sanitized output against ground-truth sensitive regions
Client resources - 20%	model bytes, JS bundle, peak memory, CPU/GPU time, inference count	Chrome DevTools + Performance API; cold/warm runs
Latency - 15%	capture, perception, privacy, network, planner, re-ground, execution; p50/p95	instrument timestamps for each stage and total task

Also track network bytes/request, task success, recovery success, unsafe-action rejection, and number of perception escalations. Do not invent target values before baseline measurements. [REQ][DESIGN]

25. Hidden / Adversarial Testing

Scenario family	Cases
Government	forms, IDs, scanned documents, OTP-like fields, PDF instructions
Banking/finance	account/card-like values, transaction confirmation, malicious page text
Email/messages	names, emails, private messages, quoted prompt injection
Healthcare	sensitive labels and document previews
Education	forms, tables, dashboards, student identifiers
E-commerce	visual products, modals, dynamic SPA, checkout-like forms
Rendering edge cases	canvas, PDF viewer, cross-origin iframe, Shadow DOM, icon-only, tiny controls, zoom/theme/viewport
Privacy adversarial	PII split across DOM nodes, OCR errors, hidden text, QR, base64-like secrets, token bait, same-value decoys
Action adversarial	layout shift after plan, duplicate buttons, disabled overlay, injected fake instruction, wrong-origin form

26. Cost Engineering

Configuration	Local cost	Cloud cost	What changes
Cheapest viable prototype	Open-source browser runtimes/models	Low hosted LLM usage	DOM+OCR+rules+token vault; no large local VLM
Competition-ready	Same + selected small CV/NER if metrics justify	Text LLM + rare safe-crop VLM	adaptive cascade, benchmarks, hardened policy
Production-grade	Model CDN/cache, broader browser QA, telemetry privacy engineering	managed planner, auth, monitoring, regional/privacy controls	multi-browser, policy management, stronger eval/ops

Exact hosted-model prices change frequently and should be captured in the final submission from the provider pricing page used at build freeze. Cost should be reported per task from measured token/image usage rather than guessed. [EXP]

27. Failure / Recovery Matrix

Failure	Detect	Recovery	Fallback
WebGPU unavailable	feature/runtime init check	switch execution provider	WASM
OCR slow/wrong	confidence + latency budget	smaller ROI/preprocess/retry	DOM text / ask user
PII uncertain	low confidence/high-risk pattern	fail closed; mask broader region	ask user
Visual model fails	no target / inconsistent box	ROI retry or safe-crop remote VLM	structure/OCR
Planner malformed	schema validation	retry with error feedback	rule action / ask user
Network/server down	timeout	local retry/backoff	local-only limited mode
Target stale	page epoch/fingerprint mismatch	re-observe and re-plan	ask user
Action has high impact	policy classification	explicit confirmation	deny
Verification fails	no expected state delta	undo if possible/re-observe	ask user

28. What NOT to Build

- Do not train a foundation model.
- Do not put a large local LLM/VLM on the critical path.
- Do not continuously run full-screen VLM inference.
- Do not build a multi-agent swarm, RAG stack, Kubernetes cluster, huge database or elaborate dashboard.

- Do not optimize Firefox/Safari before Chrome vertical-slice stability.
- Do not let the cloud emit arbitrary JavaScript or selectors.
- Do not claim privacy because screenshots are blurred; prove the exact outbound bytes.
- Do not hard-code demo-site selectors into the product path.
- Do not add a model unless it improves a scored metric or solves a demonstrated failure mode.

29. Judge Defense - High-Probability Questions

Question	Defense
Where is the computer vision?	In-browser OCR and optional local visual detectors/grounder operate on screenshot/ROI pixels before networking. Structure is used first for efficiency, not as a substitute for vision.
Why is this not DOM automation?	DOM is one sensor. Canvas/PDF/image/icon cases trigger pixel perception; the agent can ground visual-only targets.
Why not send screenshots?	They expose the largest privacy surface and waste bandwidth. We send a minimized semantic graph and only safe crops on justified escalation.
What if PII detection misses something?	Layered detectors, high-risk fail-closed policy, structural minimization, safe-crop gating, and per-class recall testing. We never claim perfect detection.
What if the server is malicious?	It never gets private-vault values and cannot execute directly. All proposals pass local schema, origin, target and policy checks.
What if the page prompt-injects the model?	Page content is tagged untrusted; user intent/policy are separate; planner has constrained actions; local authority blocks unsafe proposals.
What if the page changes before click?	Page epoch and target fingerprint are revalidated immediately before execution; stale actions are discarded.
Why lightweight?	Most steps use browser structure; expensive pixel inference is ROI/event-driven; WASM/WebGPU fallback and model budgets are measured.
What is differentiated?	The engineering contribution is privacy-minimized adaptive perception plus local authority and measurable privacy/latency/resource gates - not claiming a new foundation model.
Fallback?	Screenshot-first local redaction + cloud VLM, while preserving local policy/execution/verification.

30. Open Experiments & Final Engineering Decision

Experiment	Decision it unlocks
Tesseract vs alternative OCR on 200 UI crops	Keep/replace OCR baseline

Rules-only vs rules+small NER	Whether NER earns model cost
Face/document/QR detector browser latency	Whether visual privacy model belongs in MVP
ONNX WASM vs WebGPU on target laptops	Default execution provider and fallback
DOM/ARIA graph vs graph+OCR on unseen sites	Perception cascade threshold
GoClick or other small grounder conversion/runtime	Whether local GUI grounding is feasible
Text LLM vs VLM safe-crop escalation	When pixels need to leave device
Adversarial PII and prompt injection suite	Policy strictness and judge claims

FINAL DECISION Build the adaptive hybrid architecture. Freeze the MVP around DOM/ARIA + screenshot + local OCR + layered PII + irreversible tokenization + SafeContext + remote text planner + constrained local actions + verification. Treat NER, face/document detection, GUI grounding and remote VLM as gated upgrades. If any advanced component threatens the working vertical slice, remove it and keep the compliant fallback.

Final One-Page Decision Summary

Question	Answer
What is the problem?	Give a browser agent enough visual context to act while preventing sensitive screen data from leaving the device and staying lightweight.
What are we building?	A Chrome edge-cloud agent: local perception/privacy + remote planning + local authority/execution/verification.
Serious solution families?	Screenshot-first, structure-first, OCR-centric, vision-first, specialist, hybrid, adaptive cascade, edge-cloud, mostly local.
Top three?	#1 adaptive hybrid; #2 vision-first semantic; #3 screenshot-first redaction fallback.
Where does real vision happen?	Locally in browser workers/offscreen context on screenshot/ROI pixels.
What crosses network?	Safe semantic UI graph and rare privacy-safe crops.
What never crosses by default?	Raw screenshots, raw DOM dumps, passwords/OTP/API/session tokens, vault values, hidden values.
First 48h?	Working vertical slice with OCR, PII, real sanitized network payload, planner JSON action, local execution and verification.

Day 30?	Benchmarked, adversarially tested, stable demo with fallback build and evidence-backed claims.
Biggest technical risk?	Browser-feasible visual grounding/model integration.
Biggest privacy risk?	False negatives and leakage through non-obvious channels such as OCR/DOM/telemetry.
Biggest execution risk?	Six isolated modules integrated too late.
Cheapest viable solution?	DOM/ARIA + Tesseract + deterministic privacy rules + token vault + hosted text LLM.
Competition-ready differentiator?	Adaptive perception, minimized disclosure, local authority, and direct measurement against the official scoring dimensions.

References / Evidence Base

ID	Source
R0	Team-supplied SIH 26171 master research specification (problem boundary, scoring weights, required pipeline).
R1	Chrome for Developers - chrome.offscreen API, MV3 hidden document architecture, updated 2026-06-30. https://developer.chrome.com/docs/extensions/reference/api/offscreen
R2	Chrome for Developers - content scripts, frames and isolated worlds. https://developer.chrome.com/docs/extensions/reference/manifest/content-scripts
R3	Chrome for Developers - chrome.tabs API and captureVisibleTab; capture rate limit. https://developer.chrome.com/docs/extensions/reference/api/tabs
R4	Chrome for Developers - Stay secure; hostile pages can manipulate DOM assumptions. https://developer.chrome.com/docs/extensions/develop/security-privacy/stay-secure
R5	Li et al. ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use, 2025. https://arxiv.org/abs/2504.07981
R6	Yang et al. Ferret-UI Lite: Lessons from Building Small On-Device GUI Agents, 2025. https://arxiv.org/abs/2509.26539
R7	Wang et al. GUIGuard: Toward a General Framework for Privacy-Preserving GUI Agents, 2026. https://arxiv.org/abs/2601.18842
R8	ONNX Runtime Web - Using WebGPU. https://onnxruntime.ai/docs/tutorials/web/ep-webgpu.html
R9	ONNX Runtime Web - browser execution provider support.

	https://onnxruntime.ai/docs/get-started/with-javascript/web.html
R10	Hugging Face Transformers.js docs - in-browser ML, WASM/WebGPU and quantization. https://huggingface.co/docs/transformers.js/en/index
R11	Tesseract.js - browser/Node OCR via WebAssembly; Apache-2.0; v7 release line. https://github.com/naptha/tesseract.js
R12	Li et al. GoClick: Lightweight Element Grounding Model for Autonomous GUI Interaction, 2026. https://arxiv.org/abs/2604.23941
R13	Debenedetti et al. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents, NeurIPS 2024. https://arxiv.org/abs/2406.13352
R14	The Dangers of Indirect Prompt Injection Attacks on LLM-based Autonomous Web Navigation Agents, EMNLP 2025 demo. https://aclanthology.org/2025.emnlp-demos.55/
R15	Zhou et al. WebArena: A Realistic Web Environment for Building Autonomous Agents, 2023. https://arxiv.org/abs/2307.13854
R16	Koh et al. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks, 2024. https://arxiv.org/abs/2401.13649
R17	Deng et al. Mind2Web: Towards a Generalist Agent for the Web, 2023; dataset updates through 2025. https://github.com/OSU-NLP-Group/Mind2Web
R18	Le Sellier De Chezelles et al. The BrowserGym Ecosystem for Web Agent Research, 2024. https://arxiv.org/abs/2412.05467
R19	Li et al. Ferret-UI 2: Mastering Universal User Interface Understanding Across Platforms, 2024. https://arxiv.org/abs/2410.18967

Research cutoff: 24 Aug 2026. Any technology/model not directly benchmarked in the team browser build remains [EXP], regardless of published results.